ActionGuardian • The Popup Window Deep Dive

Comprehensive Functional, Data & Query Specification (From 5-Year-Old Analogy to Code-Level Precision)

Target Component: Section 6 Intervention Popup Modal (#v2-popup-modal)	**Spec Version:** Scoring v2 Consequentialist Model
Primary Backend Route: POST /api/preview & POST /api/rewrite	**Security Standard:** Zero-Bypass Preview-then-Commit Guarantee

1. The Story of the Magic Postman (Explained for a 5-Year-Old)

■ ***Imagine this story:***
*You have a very fast, very eager toy robot named **Robo-Postman**. Robo loves delivering letters! Whenever you write something on paper, Robo folds it into a paper airplane and throws it out the window into the sky.*

***The big problem:** Once the paper airplane flies away into the sky, nobody can ever grab it back! What if Robo wrote something mean in the letter because he was grumpy? What if Robo accidentally put Mommy's credit card or Daddy's secret password inside the envelope? Once the neighbor reads it, it's too late! You can't un-read a secret.*

Enter the Whistle Guardian (The Popup Window):
*ActionGuardian is a wise little guardian sitting right on the windowsill. Before Robo throws the airplane, the guardian catches Robo's arm and asks **3 Golden Questions**:*

1. *Why are you sending this letter? (Intent)*
2. *Could someone get mad or misunderstand? (Could it backfire?)*
3. *If it goes wrong, how bad is the boo-boo? (What is the damage?)*

*If the letter is sweet, the guardian lets it fly quietly. But if it has secrets or angry words, the guardian jumps up, blows a shiny red whistle, and shows you a **Big Warning Card** (The Popup Window) that says: 'STOP! Look at what this letter will do! Click the Magic Wand to fix it first!'*

2. When Does the Popup Wake Up? (The 4 Behavior Tiers)

ActionGuardian does not annoy users for safe, routine messages. It only opens the intervention modal when the computed risk score meets or exceeds 40. Here is how each tier behaves:

Score Range	Tier Name	Popup Behavior	5-Year-Old Explanation	Dispatch Action
0 to 39	SAFE	No Popup (Silent)	Friendly greeting! The guardian stays asleep. Letter flies away instantly.	Auto-enables the 10s Undo Send timer appears
40 to 59	MEDIUM	Yellow Popup (Small)	A little spicy or asking for money. Guardian peeks out and asks 3 questions.	Popup opens with 3 questions. Shows 'Send Anyway' c

60 to 79	HIGH	Orange Popup (Strict)	Angry tone or sensitive data. Guardian flashes bright orange warning lights.	Popup requires a manual checkbox acknowledgment. AI
80 to 100	CRITICAL	Red Modal (HARD BLOCK)	EMERGENCY! Hardcoded password or violent threat. Completely LOCKED DOWN.	Send button is completely DISABLED. Dispatch Blocked

■ **The Immutable 'Hard Floor' Rule (Spec Section 1 & 4):**
If the draft contains an **AWS key**, **OpenAI key**, **GitHub token**, **database password**, **RSA private key**, or **violent harassment**, the risk score is automatically clamped to at least **80 (CRITICAL)**. Neither prompt-engineering nor AI learning can ever lower this floor. The popup will physically refuse to dispatch until the secret is scrubbed into a safe vault token.

3. Anatomy of the Popup Window (Every DOM Element & ID)

The popup window is defined in web/index.html under container ID #v2-popup-modal. Every single visual card, badge, and button is mapped directly to live data:

DOM Element ID	Display Component	Data Source / Trigger	What the 5-Year-Old Sees
#v2-modal-icon	Icon Badge	■■ for Medium/High, ■ for Critical	Yellow caution triangle or big red stop sign.
#v2-modal-title	Modal Header	Static: 'Pre-Dispatch Safety Intervention'	'Wait a minute! Look before you leap!'
#v2-modal-tier-badge	Tier Pill	sim.risk_level ('MEDIUM', 'HIGH', 'CRITICAL')	Color tag showing how risky the message is.
#v2-modal-score	Score Counter	sim.risk_score + '/100'	Danger meter: 0 is completely safe, 100 is super dange
#v2-modal-q1	Question 1 Card	sim.v2_details.intent_summary (Indigo bar)	'Why are you writing this? (e.g., Asking for money, firin
#v2-modal-q2	Question 2 Card	sim.v2_details.backfire_summary (Amber bar)	'Could this go wrong? (e.g., Sent to external stranger, r
#v2-modal-q3	Question 3 Card	sim.v2_details.consequence_summary (Rose bar)	If it goes wrong, who gets hurt? (e.g., Company loses
#v2-modal-critical-warning	Red Alert Banner	Shown ONLY when sim.risk_score >= 80	Big red flashing box: 'Credentials found! Hard block en
#v2-modal-rewrite-btn	Rewrite Button	Calls closeV2Modal(); autoImproveEmail()	Magic Wand button! 'Fix my letter so it is safe to send.'
#v2-modal-send-btn	Send Action Button	Disabled if >=80; 'Send with Override' if 60-79	Green/Black 'Send Anyway' button, or greyed-out padl
Review in Editor	Cancel / Review	Calls closeV2Modal()	'Let me close this window and edit the words myself.'

4. The 3 Questions & Consequence Math Formula

The numbers shown inside the popup window are not random guesses or black-box hallucinations. They are computed using the exact **Scoring v2 Mathematical Formula**:

■ **The Universal Risk Equation:**
Risk Score = min(100, Content + Intent + round_half_up(Backfire_p × Impact))
(If secret_or_credential, private_key, or violent_threat is present, enforce Risk Score ≥ 80)

Building Block A: Question 1 — What is the Intent?

5-Year-Old Explanation: Why did you write this letter? Saying hello is zero danger. But talking about money or firing someone makes the danger number go up!

Intent Category	Points Added	Examples / Description
routine	0 points	Meeting schedules, status updates, friendly check-ins
complaint	5 points	Unhappy customer email, hostel mess complaint, product grievance
sensitive_info	10 points	Sharing roadmap, candidate background, internal notes
data_transfer	15 points	Sending customer database, server credentials, code files
hr_action	15 points	Firing an employee, disciplinary notice, salary revision
legal	15 points	Lawsuit threat, contract disputes, liability admissions
money	20 points	Wire transfer requests, invoices, bank account updates
intent_drift	+15 bonus	Agent did MORE than the human asked for (unprompted action)

Building Block B: Question 2 — Could It Backfire?

5-Year-Old Explanation: Who are you giving the letter to? Giving a drawing to your mom has low backfire (0.2). Yelling at 20 strangers over a loudspeaker has high backfire (0.8)!

Factor Count	Backfire Band	Probability (p)	Detected Backfire Triggers in Email
0 to 1 factor	Low Band	0.2 (20% chance)	Known internal colleague, calm tone, single recipient
2 to 3 factors	Medium Band	0.5 (50% chance)	External @gmail domain (+1), angry tone (+2), >5 recipients (+1)
4+ factors	High Band	0.8 (80% chance)	Unknown recipient (+1), lookalike domain (+2), hostile threat (+2), reply-all (+1)

Building Block C: Question 3 — If It Backfires, What Happens?

5-Year-Old Explanation: How big is the boo-boo? Spilling a drop of water on the floor is operational (15 pts). Dropping the family iPad on the concrete driveway is security (70 pts)!

Consequence Category	Base Severity	Multipliers Applied	Final Impact Capping
security	70 pts	• Scope: One = x0.8, Group = x1.0, Public = x1.2	Hard-capped at 70 maximum
legal	60 pts	• Irreversibility: x1.2 (cannot recall after delivery)	Hard-capped at 70 maximum
financial	55 pts	Example: Legal (60) × Public (1.2) × Irrev (1.2) = 86.4	Capped at 70 max pts!

reputation	50 pts	p × Impact: 0.5 × 58 = 29 points added to risk.	Added cleanly to total.

5. Complete Network Query & Data Lifecycle

Here is the exact journey of every single HTTP query, JSON payload, and backend handler from the second the user types an email to the moment the popup opens or dispatches:

Query 1: Live Recipient Genuineness Check (POST /api/verify_email)

When it fires: Automatically in the background as the user types the email address (350ms debounce).
5-Year-Old Explanation: Robo checks if the house address on the envelope actually exists in the real world before walking there.

```
// Request Payload:
{
  "email": "test@fakedomainxyz9999.com"
}

// Response Payload (404/Non-existent DNS):
{
  "is_valid": false,
  "domain_exists": false,
  "error": "No such email exists: Domain '@fakedomainxyz9999.com' does not exist on the internet.",
  "suggestion": null
}
// UI Action: Badge turns RED; dispatch button stays locked.
```

Query 2: The Simulation Scan (POST /api/preview)

When it fires: When user clicks 'Simulate & Scan for Threats' or selects a benchmark scenario.
5-Year-Old Explanation: Robo puts the letter in the safe testing scanner. The scanner figures out the score and decides whether to wake up the popup window.

```
// Request Payload:
{
  "target": "partner@external.com",
  "subject": "Production Credentials",
  "content": "Here is our master AWS key: AKIAIOSFODNN7EXAMPLE and db password: Admin@12345"
}

// Backend Processing (agent/guardian.py):
// 1. ConsequencePredictor.predict(candidate) runs rules & 3-Question math.
// 2. Secret 'AKIA...' triggers hard floor score = 100 (CRITICAL).
// 3. Sandbox diff generates isolated dry-run state.
// 4. Token cached: preview_cache['prev_a8f9c102'] = { result, diff, used: False }.

// Response Payload to Browser:
{
  "preview_id": "prev_a8f9c102",
  "simulation": {
    "risk_score": 100,
    "risk_level": "CRITICAL",
    "is_blocked": true,
    "can_auto_commit": false,
    "v2_details": {
      "intent_summary": "Intent: Data transfer (Credentials & Keys)",
      "backfire_summary": "Backfire Band: Medium (external_domain detected)",
      "consequence_summary": "Potential Impact: Security Compromise (70 pts)",
      "show_popup": true
    }
  }
}
// UI Action: showV2Modal() executes -> The Intervention Popup opens in full screen!
```

Query 3: The 1-Click AI Auto-Improve (POST /api/rewrite)

When it fires: When user clicks '■ Rewrite with AI' inside the popup modal.
5-Year-Old Explanation: You wave the magic wand! The angry words turn sweet, the secrets go into a locked bank vault, and the danger score drops to safe green numbers!

```
// Request Payload:
{
  "target": "partner@external.com",
  "subject": "Production Credentials",
  "content": "Here is our master AWS key: AKIAIOSFODNN7EXAMPLE and db password: Admin@12345",
  "iteration": 1
}

// Backend Action (agent/predictor.py -> rewrite_email_safely):
// - Scrubs 'AKIA...' -> 'Centralized Secret Manager (Vault Ref #SEC-4091)'.
// - Scrubs 'Admin@12345' -> 'SSO Bastion Authentication'.
// - Recalibrates score: 100 (CRITICAL) -> drops to 28-35 (SAFE zone)!

// Response Payload:
{
  "remediated_subject": "Production Credentials [SECURE ACCESS]",
  "remediated_content": "Hi Team,\n\nAccess has been provisioned via enterprise Vault #SEC-4091...",
  "old_score": 100,
  "new_score": 28,
  "reduction": 72,
  "clean_preview": { "preview_id": "prev_clean_4b81", "risk_score": 28, "risk_level": "SAFE" }
}
// UI Action: closeV2Modal(); Email editor updates; Badge turns GREEN; Send unlocked!
```

Query 4: Final Security Commit & Real Dispatch (POST /api/send)

When it fires: When user clicks 'Send Real Email' (or 'Send Anyway' from Medium tier).
5-Year-Old Explanation: The guardian checks the ticket. The ticket is signed and stamped. The letter is put into the real post office box and delivered!

```
// Request Payload:
{
  "preview_id": "prev_clean_4b81",
  "recipient": "verified.partner@gmail.com",
  "subject": "Production Credentials [SECURE ACCESS]",
  "content": "Hi Team, access provisioned via Vault #SEC-4091...",
  "user_override": false
}

// Security Verification in app.py:
// - Validates 'prev_clean_4b81' in preview_cache (Rejects direct unsimulated attempts with 403).
// - Invalidator sets used = True (Prevents replay attacks).
// - smtplib sends via smtp.gmail.com:587 with TLS encryption.

// Response Payload:
{
  "status": "SENT",
  "recipient": "verified.partner@gmail.com",
  "receipt_id": "rcpt_7f309a1b",
  "timestamp": 1730000000,
  "audit_message": "Email dispatched successfully via Gmail SMTP with TLS encryption."
}
```

6. Live Judge Testing Scenarios: Abrupt vs. Nice Emails in Action

During live hackathon demonstrations, judges are invited to type raw, arbitrary communications into the composer. The table below documents how ActionGuardian dynamically differentiates between abrupt/slang/toxic emails and polite, easy-going inquiries:

Sample Input Category	Exact Text Tested	Score	Tier	Popup?	System Action & AI Remedy
Slang / Swear	'What the fuck are you doing? This is bullsh...	84	CRITICAL	YES (Block)	Slams hard block. AI rewrite neutralizes profanity to fo
Direct Insult	'You idiot, fix this damn code right now!'	84	CRITICAL	YES (Block)	Detects severe hostility. Replaces abuse with structure
Indian Slang	'Saale bakwas band kar, you are useless.'	84	CRITICAL	YES (Block)	Multi-lingual colloquial detector catches regional slang
Legal Lawsuit	'I will sue you in court if you do not pay me!'	100	CRITICAL	YES (Block)	Classifies as legal dispute. Rewrites to formal settleme
Emergency Wire	'Wire 50000 dollars to my bank account immediately.'	100	CRITICAL	YES (Block)	Flags financial wire patterns and unauthorized fund dis
Employment Fired	'You are fired effective immediately. Pack your things.'	94	CRITICAL	YES (Block)	Enforces statutory HR compliance; transforms unvette
Severe Complaint	'This service is completely unacceptable and terrible.'	60	HIGH	YES (Review)	Elevated complaint tier; requires human acknowledgm
Friendly Nice Email	'Hi team, can we move our sync to 3 PM? Thanks!'	2	SAFE	NO (Silent)	Zero false alarms. Passes cleanly with 10-second silen
Polite Follow-up	'Good morning Sarah, just following up on the proposal.'	2	SAFE	NO (Silent)	Routine communication cleared immediately with zero

■ Final Architectural Summary:
The ActionGuardian Popup Window is not a cosmetic notification. It is the physical manifestation of the **Preview-then-Commit Gateway**. It mathematically translates content semantics, recipient DNS validity, and contextual blast radius into an actionable human decision: **Empower the user with self-healing AI, or block catastrophic actions before network dispatch.**